

Vérification statique des séquences d'appels aux fonctions collectives MPI

COAV35 – Projet

LORENZO LUCAS -- ROBLOT ALICIA PLATH

Vendredi 21 novembre 2025

1. Présentation du projet
2. Passe de compilation et rang
3. Frontière de post-dominance
4. Gestion des directives
5. Perspectives
6. Démonstration

Présentation du projet

GCC

- GNU Compiler Collection
- C, C++, Fortran
- Système de **plugins** pour ajouter/modifier des passes de compilation

Objectifs

- Réalisation d'un plugin [1] afin de détecter les deadlocks
- Analyse des positions des collectives MPI [2]
- Affichage d'un warning en cas de deadlock
- Gestion des directives (`#pragma`)

Exemple de situation de deadlock

```
int main(int argc, char * argv[])
{
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        MPI_Barrier(MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 1;
}
```

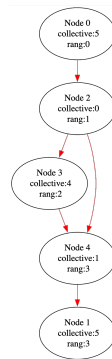


Figure : CFG associé à un deadlock

Passe de compilation et rang

Création de la passe – plugin.h

```
int plugin_is_GPL_compatible;
int plugin_init(struct plugin_name_args *plugin_info, struct plugin_gcc_version *version);
const pass_data my_pass_data {
    GIMPLE_PASS, "my_pass", OPTGROUP_NONE, TV_NONE, PROP_gimple_any, 0, 0, 0, 0,
};
class my_pass : public gimple_opt_pass {
public:
    my_pass(gcc::context *ctxt) : gimple_opt_pass (my_pass_data, ctxt) {}
    my_pass* clone() { return new my_pass(g); }
    bool gate(function *fun);
    unsigned int execute(function *fun);
};
```

Intérêt de la passe

- Trouver les appels aux fonctions collectives MPI
- Bibliothèque dynamique libplugin.so

Notions

- **CFG** – Graphe de flot de contrôle, représentation du flux d'exécution d'un programme
- **Bloc de base** – Séquence d'instructions consécutives
- **Rang** – Nombre maximal de collectives MPI rencontrées depuis le début du programme

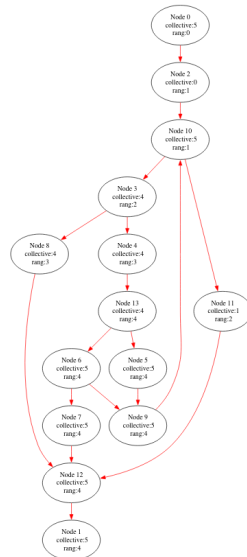
Associer un rang à chaque collective MPI

- Partitionner les blocs de base comportant plusieurs appels MPI (`splitBlock`)
- Enregistrement et identification des appels MPI (`saveCollective`)
- Calculer le rang de chaque bloc de base en ignorant les boucles du CFG (exemple dans l'[annexe](#))

Frontière de post-dominance

– Listing 1 : test3.c –

```
int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);
    int i; int a = 2; int b = 3; int c=0;
    for (i=0; i<10; i++) {
        MPI_Barrier(MPI_COMM_WORLD);
        if (c<10) {
            MPI_Barrier(MPI_COMM_WORLD);
            MPI_Barrier(MPI_COMM_WORLD);
            if (c <5) a = a*a + 1;
            else {
                c = c*3;
                if (c <20) return c;
            }
        }
        else MPI_Barrier(MPI_COMM_WORLD); return 1;
        c+= (a+b);
    }
    MPI_Finalize();
    return 1;
}
```



Formules

$$\text{PDF}(E) = \{X \mid \exists S \in \text{succs}(X), E \gg_p S \text{ et } E \not\gg_p X\}$$
$$E \gg_p X \Leftrightarrow \forall S \in \text{succs}(X), \exists Y \subset E, Y \gg_p S$$

– Listing 2 : doesEPostDomineX –

```
bool doesEPostDomineX(function* fun, bitmap_head* e,  
↳ basic_block x, bitmap_head visited) {  
    if (bitmap_bit_p(e, x->index)) return true;  
    if (EDGE_COUNT(x->succs) == 0) return false;  
    if (bitmap_bit_p(&visited, x->index)) return true;  
  
    edge p;  
    edge_iterator ei;  
    FOR_EACH_EDGE(p, ei, x->succs) {  
        bitmap_set_bit(&visited, x->index);  
        if (!doesEPostDomineX(fun, e, p->dest,  
↳ visited)) return false;  
    }  
    return true;  
}
```

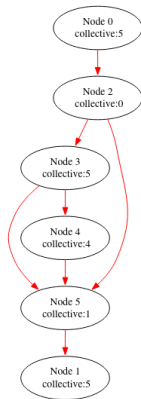


Figure : CFG du programme test_simple2.c

– Listing 3 :frontierePostDominanceIteree –

```
void frontierePostDominanceIteree(function* fun,
↳ bitmap_head* pdf, bitmap_head* pdf_iter) {
    bool isPdfEmpty = false;
    while (!isPdfEmpty && !bitmap_intersect_p((const
↳ bitmap)pdf_iter, (const bitmap)pdf)) {
        bitmap_head copy;
        bitmap_initialize(&copy,
↳ &bitmap_default_obstack);
        bitmap_copy(&copy, pdf_iter);
        bitmap_ior(pdf_iter, (const_bitmap)&copy,
↳ (const_bitmap)pdf);

        bitmap_head pdf_of_pdf ;
        bitmap_initialize (&pdf_of_pdf,
↳ &bitmap_default_obstack);
        isPdfEmpty = frontierePostDominance(fun, pdf,
↳ &pdf_of_pdf);

        if(!isPdfEmpty) bitmap_copy(pdf, &pdf_of_pdf);
    }
}
```

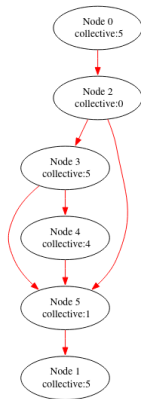


Figure : CFG du programme test_simple2.c

Gestion des directives

- `#pragma ProjetCA mpicoll_check mafunc`
- `#pragma ProjetCA mpicoll_check(mafunc1,mafunc2,mafunc3)`



```
c_register_pragma("ProjetCA", "mpicoll_check", my_pragma_handler);
```

– Listing 4 : Fichier pragma.cpp –

```
1  #include "pragma.h"
2  #include <tree.h>
3  #include <diagnostic.h>
4
5  vec<const char*> list_func;
6  bool pragma_execute = false;
7
8  void my_pragma_handler(cpp_reader *dummy){
9
10     location_t loc;
11     enum cpp_ttype token;
12     tree x;
13     bool close_paren_needed_p = false;
14
15     if (cfun){
16         error ("%<#pragma GCC option%> is not allowed inside functions");
17         return;
18     }
19
20     token = pragma_lex (&x, &loc);
21     if (token == CPP_OPEN_PAREN){
22         close_paren_needed_p = true;
23         token = pragma_lex(&x, &loc);
24     }
```

– Listing 5 : Fichier pragma.cpp –

```
1  if (token == CPP_NAME){
2      do{
3          if (!list_func.contains(IDENTIFIER_POINTER(x)))
4              ↪ list_func.safe_push(IDENTIFIER_POINTER(x));
5
6          else warning_at(loc, 0, "The function name: %s appears more than
7              ↪ 1 time", IDENTIFIER_POINTER(x));
8
9          token = pragma_lex(&x);
10         while (token == CPP_COMMA) token = pragma_lex(&x);
11     } while (token == CPP_NAME);
12
13     if (close_paren_needed_p){
14         if (token == CPP_CLOSE_PAREN) token = pragma_lex (&x);
15         else error ("%<#pragma GCC target (string [,string]...)%> does
16             ↪ not have a final %>%>");
17     }
18
19     if (token != CPP_EOF){
20         error ("%<#pragma GCC target%> string is badly formed");
21         return;
22     }
23     pragma_execute = true;
24 }
```

>_

```
test_simple.c:13:13: erreur: « #pragma GCC option » n'est
pas permis à l'intérieur des fonctions
```

```
13 | #pragma ProjetCA mpicoll_check main
    | ~~~~~
```

>_

```
test_simple.c:7:33: attention: The function name: main
appears more than 1 time
```

```
7 | #pragma ProjetCA mpicoll_check (main, main)
  | ~~~~~
```

Traitement ou non ? – plugin.cpp

```
bool my_pass::gate(function *fun) {
    if (pragma_execute) {
        const char* fname =
            ↪ fndecl_name(cfun->decl);

        if (list_func.contains(fname)) {
            used_func.safe_push(fname);
            return true;
        }
        return false;
    }
    return true;
}
```


Perspectives

 – Listing 6 : Extrait du fichier `post_dominance.cpp` –

```
1  tree type = build_function_type(void_type_node, void_list_node);
2  tree fndecl1 = build_fn_decl("branche1", type);
3  TREE_PUBLIC(fndecl1) = 1;
4  DECL_EXTERNAL(fndecl1) = 1;
5
6  ...
7
8  gimple* call;
9  call = gimple_build_call(fndecl1, 0);
10 gsi_insert_before(&gsi, call, GSI_SAME_STMT);
```

Démonstration

Annexe

– Listing 7 : test_simple.c –

```
int main(int argc, char * argv[]) {  
    MPI_Init(&argc, &argv);  
  
    int rank;  
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
  
    if (rank == 0) {  
        MPI_Barrier(MPI_COMM_WORLD);  
    }  
  
    MPI_Finalize();  
    return 1;  
}
```

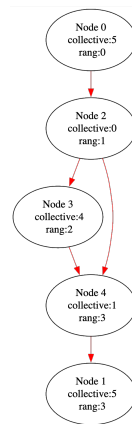


Figure : CFG du programme test_simple.c

– Listing 8 : test_simple2.c –

```
int main(int argc, char * argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    int i = 0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank %2 ==0) {
        i += 1;
        if (rank == 0)
            ↪ MPI_Barrier(MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return i;
}
```

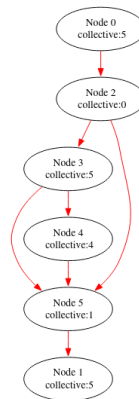


Figure : CFG du programme test_simple2.c

– Listing 9 : test_simple3.c –

```
int main(int argc, char * argv[]) {  
    MPI_Init(&argc, &argv);  
  
    int rank;  
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
    if (rank == 0) {  
        MPI_Barrier(MPI_COMM_WORLD);  
    }  
    else {  
        MPI_Barrier(MPI_COMM_WORLD);  
    }  
  
    MPI_Finalize();  
    return 1;  
}
```

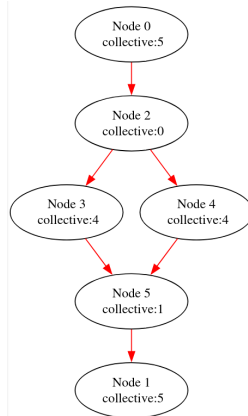


Figure : CFG du programme test_simple3.c

– Listing 10 : test_simple4.c –

```
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int value = rank + 1; int sum = 0;
    if (rank == 0) {
        ↪ MPI_Barrier(MPI_COMM_WORLD);
        ↪ MPI_Reduce(...); }
    else { MPI_Reduce(...);
        ↪ MPI_Barrier(MPI_COMM_WORLD); }

    MPI_Finalize();
    return 0;
}
```

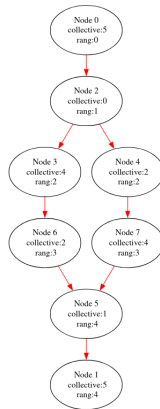


Figure : CFG du programme test_simple4.c

– Listing 11 : test_simple5.c –

```
#pragma ProjetCA mpicoll_check main

int main(int argc, char * argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    int i=0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    for (int i=0; i<rank; i++) {
        MPI_Barrier(MPI_COMM_WORLD);
    }
    MPI_Barrier(MPI_COMM_WORLD);

    MPI_Finalize();
    return i;
}
```

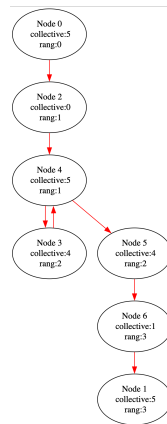


Figure : CFG du programme test_simple5.c

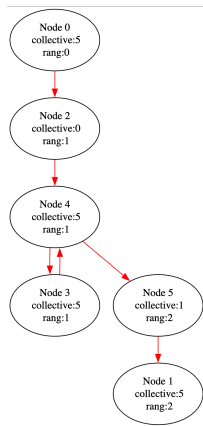


Figure : CFG du programme `test2.c`

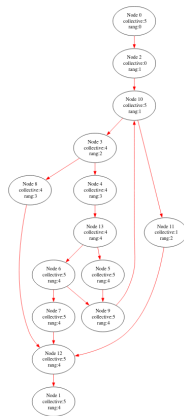
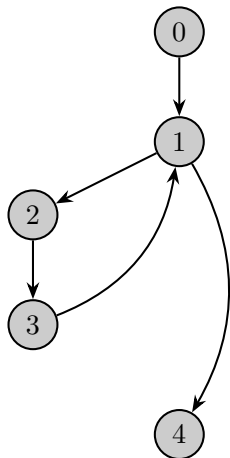
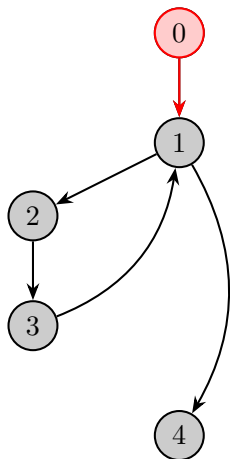


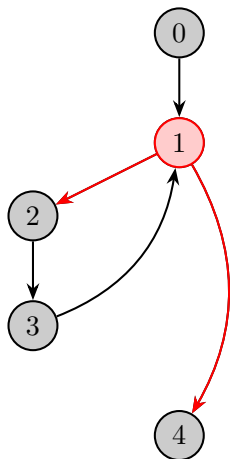
Figure : CFG du programme `test3.c`



0	<table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>											EDGES PATH
1	<table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>											EDGES PATH
2	<table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>											EDGES PATH
3	<table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>											EDGES PATH
4	<table><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>											EDGES PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	PATH
1						EDGES
						PATH
2						EDGES
						PATH
3						EDGES
						PATH
4						EDGES
						PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2						EDGES
3						EDGES
4						EDGES

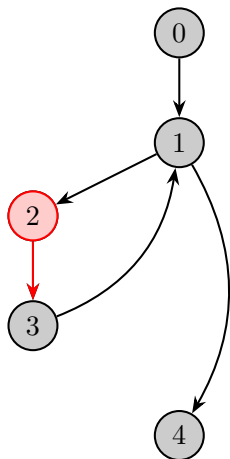
PATH

PATH

PATH

PATH

PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3						EDGES
4						EDGES

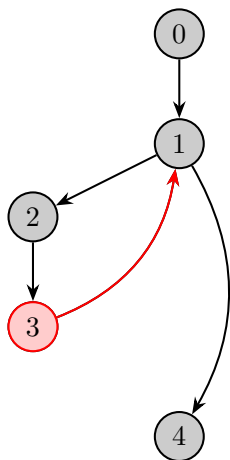
PATH

PATH

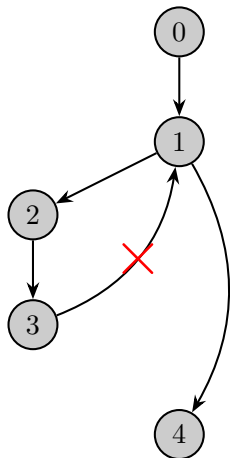
PATH

PATH

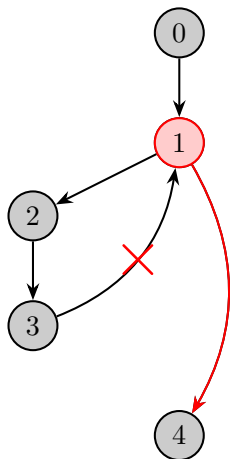
PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3	0	1	0	0	0	EDGES
	1	1	1	1	0	
4						EDGES



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3	0	1	0	0	0	EDGES
	1	1	1	1	0	
4	0	0	0	0	0	EDGES
	1	1	0	0	1	



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3	0	1	0	0	0	EDGES
	1	1	1	1	0	
4	0	0	0	0	0	EDGES
	1	1	0	0	1	

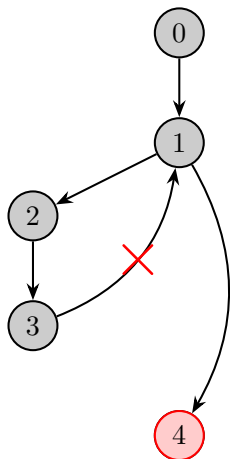
PATH

PATH

PATH

PATH

PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3	0	1	0	0	0	EDGES
	1	1	1	1	0	
4	0	0	0	0	0	EDGES
	1	1	0	0	1	

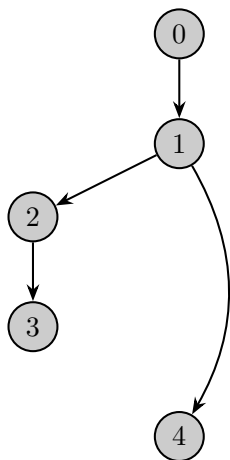
PATH

PATH

PATH

PATH

PATH



0	0	1	0	0	0	EDGES
	1	0	0	0	0	
1	0	0	1	0	1	EDGES
	1	1	0	0	0	
2	0	0	0	1	0	EDGES
	1	1	1	0	0	
3	0	1	0	0	0	EDGES
	1	1	1	1	0	
4	0	0	0	0	0	EDGES
	1	1	0	0	1	

PATH

PATH

PATH

PATH

PATH



Free Software Foundation.

Plugins – GNU Compiler Collection (GCC) Internals.

`https:`

`//gcc.gnu.org/onlinedocs/gcc-12.2.0/gccint/Plugins.html#Plugins.`



Message Passing Interface Forum.

MPI: A Message-Passing Interface Standard Version 5.0, June 2025.